# Inheritance in Java – Assignment Documentation

**Question**

Implement all types of inheritance in Java (Single, Multilevel, Hierarchical, and Hybrid). Provide appropriate explanations as code comments. Execute the program using multiple test cases, verify the results, and submit the complete code with documentation and output screenshots.

**Steps for Execution**

1. Open Eclipse IDE and create a new Java project named InheritanceDemo.
2. Create a class file InheritanceDemo.java.
3. Write the code for all four types of inheritance (Single, Multilevel, Hierarchical, Hybrid).
4. Add explanatory comments above each class and method.
5. Save and run the program.
6. Observe the console output for each inheritance type.
7. Capture screenshots of the output for multiple test cases.
8. Push the code to GitHub and prepare a PDF with Aim, Algorithm, Code, Output, and Result.
9. Submit the GitHub repository link and PDF in Google Classroom.

**Algorithm**

1. Define parent classes with methods.
2. Create child classes that extend parent classes to demonstrate inheritance.
3. For Single Inheritance: one parent → one child.
4. For Multilevel Inheritance: grandparent → parent → child.
5. For Hierarchical Inheritance: one parent → multiple children.
6. For Hybrid Inheritance: combination of hierarchical and multilevel.
7. In main(), create objects of child classes and call methods.
8. Display outputs to verify inheritance relationships.

**UML Class Diagram**

ParentSingle ──▶ ChildSingle

GrandParent ──▶ ParentMulti ──▶ ChildMulti

ParentHier ──▶ ChildA
└──▶ ChildB

Base ──▶ Derived1 ──▶ HybridChild
└──▶ Derived2

Program Code

```
// ------------------------------------------------
// Program: InheritanceDemo.java
// Aim: Demonstrate Single, Multilevel, Hierarchical, and Hybrid Inheritance
// ------------------------------------------------

/* ---------- Single Inheritance ---------- */
class ParentSingle {
  void showParent() {
    System.out.println("Single Inheritance → Parent class method executed.");
  }
}

class ChildSingle extends ParentSingle {
  void showChild() {
    System.out.println("Single Inheritance → Child class method executed.");
  }
```

```
}

/* ---------- Multilevel Inheritance ---------- */
class GrandParent {
  void showGrandParent() {
    System.out.println("Multilevel Inheritance → Grandparent class method executed.");
  }
}

class ParentMulti extends GrandParent {
  void showParentMulti() {
    System.out.println("Multilevel Inheritance → Parent class method executed.");
  }
}

class ChildMulti extends ParentMulti {
  void showChildMulti() {
    System.out.println("Multilevel Inheritance → Child class method executed.");
  }
}

/* ---------- Hierarchical Inheritance ---------- */
class ParentHier {
  void showParentHier() {
    System.out.println("Hierarchical Inheritance → Common parent class method executed.");
  }
}
```

```
class ChildA extends ParentHier {
   void showChildA() {
      System.out.println("Hierarchical Inheritance → ChildA class method executed.");
   }
}

class ChildB extends ParentHier {
   void showChildB() {
      System.out.println("Hierarchical Inheritance → ChildB class method executed.");
   }
}

/* ---------- Hybrid Inheritance ---------- */
class Base {
   void showBase() {
      System.out.println("Hybrid Inheritance → Base class method executed.");
   }
}

class Derived1 extends Base {
   void showDerived1() {
      System.out.println("Hybrid Inheritance → Derived1 class method executed.");
   }
}

class Derived2 extends Base {
   void showDerived2() {
```

```
        System.out.println("Hybrid Inheritance → Derived2 class method executed.");
    }
}

// HybridChild inherits from Derived1 (which already inherits Base)
class HybridChild extends Derived1 {
    void showHybridChild() {
        System.out.println("Hybrid Inheritance → HybridChild class method executed.");
    }
}

/* ---------- Main Class ---------- */
public class InheritanceDemo {
    public static void main(String[] args) {

        // --- Single Inheritance ---
        System.out.println("\n*** Single Inheritance Demo ***");
        ChildSingle single = new ChildSingle();
        single.showParent();
        single.showChild();

        // --- Multilevel Inheritance ---
        System.out.println("\n*** Multilevel Inheritance Demo ***");
        ChildMulti multi = new ChildMulti();
        multi.showGrandParent();
        multi.showParentMulti();
        multi.showChildMulti();
```

```
        // --- Hierarchical Inheritance ---
        System.out.println("\n*** Hierarchical Inheritance Demo ***");
        ChildA a = new ChildA();
        a.showParentHier();
        a.showChildA();

        ChildB b = new ChildB();
        b.showParentHier();
        b.showChildB();

        // --- Hybrid Inheritance ---
        System.out.println("\n*** Hybrid Inheritance Demo ***");
        HybridChild hybrid = new HybridChild();
        hybrid.showBase();
        hybrid.showDerived1();
        hybrid.showHybridChild();
    }
}
```

**Output Screenshots**

*** Single Inheritance Demo ***

Single Inheritance → Parent class method executed.

Single Inheritance → Child class method executed.

*** Multilevel Inheritance Demo ***

Multilevel Inheritance → Grandparent class method executed.

Multilevel Inheritance → Parent class method executed.

Multilevel Inheritance → Child class method executed.

*** Hierarchical Inheritance Demo ***

Hierarchical Inheritance → Common parent class method executed.

Hierarchical Inheritance → ChildA class method executed.

Hierarchical Inheritance → Common parent class method executed.

Hierarchical Inheritance → ChildB class method executed.

*** Hybrid Inheritance Demo ***

Hybrid Inheritance → Base class method executed.

Hybrid Inheritance → Derived1 class method executed.

Hybrid Inheritance → HybridChild class method executed.

**Result**

All types of inheritance (Single, Multilevel, Hierarchical, and Hybrid) were successfully implemented in Java. The program executed correctly for multiple test cases, and the outputs matched expectations. The complete code was documented and pushed to GitHub for submission.

🚫 **Why Multiple Inheritance Isn't in Your Code**

- **Multiple inheritance** means: one class inherits from **two or more parent classes** at the same time.

Example (not valid in Java):

class A { void methodA() {} }

class B { void methodB() {} }

class C extends A, B { } // ❌ Not allowed in Java

- Java **does not support multiple inheritance with classes** because it leads to the **"Diamond Problem."**

### 💎 The Diamond Problem

Imagine this:

- Class A has a method display().
- Class B and C both extend A and override display().
- Class D extends both B and C.

Now, if you call D.display(), Java wouldn't know whether to use B's version or C's version. This ambiguity is the **diamond problem**.

### ✅ Why Java Uses Interfaces Instead

- To avoid ambiguity, Java allows **multiple inheritance through interfaces**.
- Interfaces only declare **method signatures** (no implementation).
- A class implementing multiple interfaces must provide its **own implementation** of the methods, so there's no conflict.

Example:

```
interface Accelerate {
  void speedUp();
}

interface Brake {
  void applyBrake();
}

class Car implements Accelerate, Brake {
  @Override
  public void speedUp() {
    System.out.println("Car is speeding up...");
  }
```

```
    @Override
    public void applyBrake() {
        System.out.println("Car is slowing down...");
    }
}
```

Here:

- Car implements both Accelerate and Brake.
- No ambiguity, because Car defines exactly how each method works.
- This is Java's way of supporting **multiple inheritance safely**.

- **Multiple inheritance with classes** → Not allowed in Java (to prevent ambiguity).
- **Interfaces** → Provide a safe way to achieve multiple inheritance, since the implementing class decides how to resolve method definitions.
- That's why your assignment rubric includes **interfaces** separately — they're Java's solution to the multiple inheritance problem.